Week 5 – Prediction of Numerical Values (Regression)

Workflow: **problem → numerical target → features → train/test split → preprocessing → baseline → model → predictions → errors → validation → interpretation**

Tuesday – Concepts

- **Regression** predicts a **number** (e.g. the next quiz score), while classification predicts a category.
- **Observation / target / features:** a row, the number to predict (y), and the information known *before* the prediction point (X).
- **Residual** = actual − predicted. For example, actual 80 and predicted 75 gives a residual of +5.
- **Linear regression (VanderPlas, Zaki & Meira ch. 23):** prediction = intercept + $\text{coef}_1 \cdot x_1$ + $\text{coef}_2 \cdot x_2$ + ... The model learns the intercept and coefficients with `fit()` and makes predictions with `predict()`. Multiple regression uses several features, so it cannot be drawn in 2D.
- **Coefficient:** the change in the prediction when one feature increases by 1 unit and the other features stay the same.
- **Ridge regression:** still linear, but it adds a penalty that keeps the coefficients small (regularization).
- More features do not automatically make a better model. They can add noise or overfit.

Short practical check

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

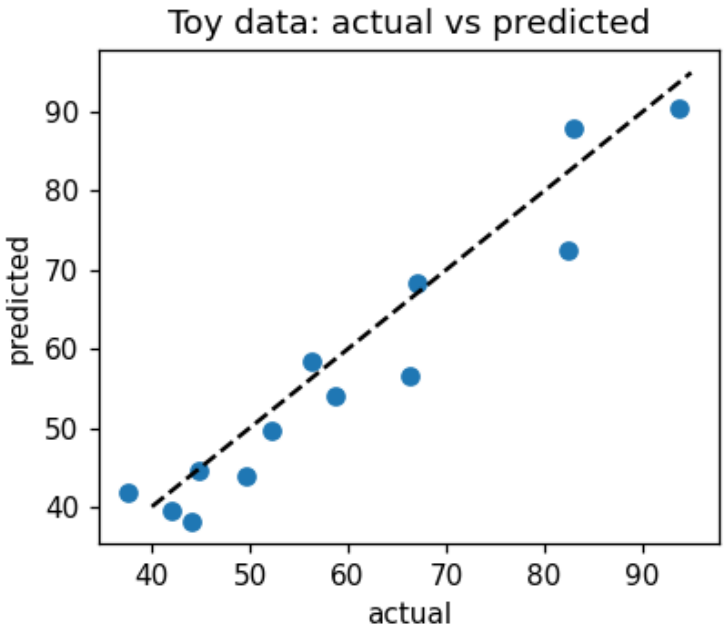
rng = np.random.default_rng(0)
hours = rng.uniform(0, 10, 50)
score = 40 + 5 * hours + rng.normal(0, 5, 50)          # small toy dataset
X_toy, y_toy = hours.reshape(-1, 1), score

Xtr, Xte, ytr, yte = train_test_split(X_toy, y_toy, test_size=0.25, random_state=1)
toy_model = LinearRegression().fit(Xtr, ytr)
pred = toy_model.predict(Xte)
print("intercept:", round(toy_model.intercept_, 1), " slope:", round(toy_model.coef_[0], 2))
display(pd.DataFrame({"actual": yte, "predicted": pred, "residual": yte - pred}).round(1).head())

plt.figure(figsize=(4, 3.2)); plt.scatter(yte, pred); plt.plot([40, 95], [40, 95], "k--")
plt.xlabel("actual"); plt.ylabel("predicted"); plt.title("Toy data: actual vs predicted"); plt.show()
```

```
intercept: 37.3  slope: 5.43
```

	actual	predicted	residual
0	93.7	90.5	3.2
1	58.7	54.1	4.6
2	67.1	68.3	-1.2
3	82.9	88.0	-5.1
4	42.0	39.5	2.5



The model found an intercept of about 40 and a slope of about 5, which is close to how the toy data was made. The points lie near the dashed line where predicted = actual.

Wednesday – Part A: Understand the prediction problem

In [2]:
```
df = pd.read_csv("synthetic-regression-dataset.csv")
print(df.shape, "| students:", df["student_key"].nunique(), "| weeks:", df["course_week"].nunique())
display(df.head(3))
df.isna().sum()[lambda s: s > 0]
```

```
(280, 17) | students: 40 | weeks: 7
```

	student_key	course_week	group_code	current_quiz_score_pct	previous_task_score_pct	concept_map_score_pct	monday_activity_count	task_attempts	attendance_recorded	task_submitted	submission_delay_minutes	submission_timing	project_activity_count	prior_average_quiz_pct	cumulative_attendance_rate	next_week_quiz_grade_0_5	next_week_quiz_score_pct
0	SYN001	1	G06	43.2	53.9	56.3	19	1	1	0	NaN	not_submitted	2.0	43.2	1.000	1	38.6
1	SYN001	2	G06	38.6	35.5	52.6	14	0	0	1	-84.2	early	2.0	40.9	0.500	2	43.4
2	SYN001	3	G06	43.4	49.2	53.9	22	1	1	1	85.3	late	0.0	41.7	0.667	2	47.9

```
current_quiz_score_pct      14
previous_task_score_pct     22
concept_map_score_pct       20
submission_delay_minutes    61
project_activity_count       8
dtype: int64
```

1. **Observation:** one student in one course week (40 students × 7 weeks = 280 rows).
2. **Numerical target:** `next_week_quiz_score_pct`, the quiz score (0–100 %) in the **following** week.
3. **Prediction point:** the end of the current week, before next week's quiz.
4. **Available before that point:** current quiz score, previous task score, concept map score, Monday activity, task attempts, attendance, task submitted / submission timing, project activity, prior average quiz score and cumulative attendance rate.
5. **Future information (leakage):** `next_week_quiz_grade_0_5` is simply the target converted into a grade from 0 to 5, so it must not be used.
6. **Identifiers:** `student_key`, `group_code` (and `course_week` as a time index).
7. **Missing values:** yes, in current quiz score (14), previous task score (22), concept map score (20), project activity (8) and submission delay (61). The delay is missing exactly when the task was not submitted, so I use `submission_timing` instead.
8. **Large error:** e.g. predicting 75 % for a student who actually gets 45 %. If the prediction were used to offer support, that student could be missed.

Part B: Create X and y

In [3]:
```
target = "next_week_quiz_score_pct"
num_features = ["current_quiz_score_pct", "previous_task_score_pct", "concept_map_score_pct",
                "monday_activity_count", "task_attempts", "attendance_recorded", "task_submitted",
                "project_activity_count", "prior_average_quiz_pct", "cumulative_attendance_rate"]
cat_features = ["submission_timing"]

X = df[num_features + cat_features]
y = df[target]
print("X:", X.shape, " y:", y.shape)
print("Categories of submission_timing:", sorted(df["submission_timing"].unique()))
```

```
X: (280, 11)  y: (280,)
Categories of submission_timing: ['early', 'late', 'near_deadline', 'not_submitted']
```

I use 10 numerical features and 1 categorical feature. Not used: the identifiers, `course_week`, `submission_delay_minutes` (replaced by the timing category) and `next_week_quiz_grade_0_5` (leakage). Preprocessing: median imputation for missing numbers, scaling, and one-hot encoding for `submission_timing`.

Part C: Split before fitting

In [4]:
```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
print("train:", X_train.shape[0], " test:", X_test.shape[0])

prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())]), num_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_features)])
```

```
train: 210  test: 70
```

Part D: Baseline

In [5]:
```
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

baseline = Pipeline([("prep", prep), ("model", DummyRegressor(strategy="mean"))]).fit(X_train, y_train)
b_pred = baseline.predict(X_test)
print("Baseline always predicts:", round(b_pred[0], 1))
print("Baseline MAE:", round(mean_absolute_error(y_test, b_pred), 2))
```

```
Baseline always predicts: 66.8
Baseline MAE: 11.62
```

Part E: Linear regression

In [6]:
```
linreg = Pipeline([("prep", prep), ("model", LinearRegression())]).fit(X_train, y_train)
lr_pred = linreg.predict(X_test)

display(pd.DataFrame({"actual": y_test.values, "predicted": lr_pred, "residual": y_test.values -
lr_pred}).round(1).head(8))

coefs = pd.Series(linreg.named_steps["model"].coef_, index=linreg.named_steps["prep"].get_feature_names_out())
print("Intercept:", round(linreg.named_steps["model"].intercept_, 1))
coefs.round(2).sort_values()
```

	actual	predicted	residual
0	91.7	81.9	9.8
1	58.4	55.3	3.1
2	63.5	68.8	-5.3
3	75.5	65.4	10.1
4	53.2	59.5	-6.3
5	45.1	54.1	-9.0
6	58.4	52.1	6.3
7	64.5	67.8	-3.3

```
Intercept: 66.9
```

```
cat__submission_timing_near_deadline    -1.12
num__task_submitted                     -0.29
num__project_activity_count             -0.07
num__attendance_recorded                -0.04
cat__submission_timing_late              0.00
num__cumulative_attendance_rate          0.11
cat__submission_timing_not_submitted     0.12
num__monday_activity_count               0.49
num__previous_task_score_pct             0.54
cat__submission_timing_early             1.00
num__task_attempts                       1.00
num__concept_map_score_pct               1.19
num__current_quiz_score_pct              2.05
num__prior_average_quiz_pct             10.19
dtype: float64
```

The numerical features are standardized, so each coefficient means *the change in predicted score (percentage points) for a 1 standard deviation increase in that feature, with the other features held constant*. - The largest coefficients are for **prior average quiz score** and **current quiz score**, which makes sense because earlier quiz results are the best guide to the next quiz. - Most other coefficients are small (under about 1 point). Current and prior quiz scores are strongly correlated (0.92), so they share the effect, and the prior average takes most of it. Coefficients show association, not cause.

Part F: Ridge regression

In [7]:
```
from sklearn.linear_model import Ridge
ridge = Pipeline([("prep", prep), ("model", Ridge(alpha=1.0))]).fit(X_train, y_train)
r_pred = ridge.predict(X_test)
pd.DataFrame({"LinearRegression": coefs,
              "Ridge": ridge.named_steps["model"].coef_}).round(2)
```

	LinearRegression	Ridge
num__current_quiz_score_pct	2.05	2.26
num__previous_task_score_pct	0.54	0.56
num__concept_map_score_pct	1.19	1.24
num__monday_activity_count	0.49	0.48
num__task_attempts	1.00	0.98
num__attendance_recorded	-0.04	-0.05
num__task_submitted	-0.29	-0.29
num__project_activity_count	-0.07	-0.04
num__prior_average_quiz_pct	10.19	9.88
num__cumulative_attendance_rate	0.11	0.13
cat__submission_timing_early	1.00	0.98
cat__submission_timing_late	0.00	-0.01
cat__submission_timing_near_deadline	-1.12	-1.09
cat__submission_timing_not_submitted	0.12	0.12

Ridge gives almost the same coefficients, only slightly smaller. With 11 features and 210 training rows, the ordinary model is not overfitting much, so the regularization changes little.

Thursday – 1–2. Evaluate the models

In [8]:
```
def scores(name, pred, note):
    return {"Model": name, "MAE": round(mean_absolute_error(y_test, pred), 2),
            "RMSE": round(root_mean_squared_error(y_test, pred), 2), "R²": round(r2_score(y_test, pred), 3),
            "Main observation": note}
results = pd.DataFrame([
    scores("DummyRegressor", b_pred, "always predicts the training mean"),
    scores("LinearRegression", lr_pred, "clearly better than the baseline"),
    scores("Ridge", r_pred, "practically the same as LinearRegression")])
results
```

	Model	MAE	RMSE	R²	Main observation
0	DummyRegressor	11.62	13.72	-0.100	always predicts the training mean
1	LinearRegression	6.88	8.35	0.592	clearly better than the baseline
2	Ridge	6.89	8.36	0.591	practically the same as LinearRegression

In [9]:
```
abs_err = np.abs(y_test.values - lr_pred)
print("Largest absolute errors (LinearRegression):", np.sort(abs_err)[-5:].round(1))
print("Share of test rows with error > 15 points:", round((abs_err > 15).mean(), 3))
```

```
Largest absolute errors (LinearRegression): [14.6 17.  17.3 18.1 19. ]
Share of test rows with error > 15 points: 0.057
```

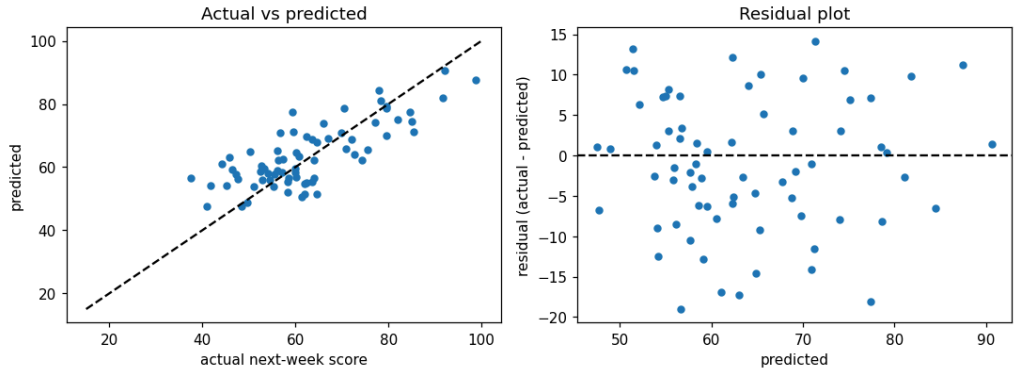
1. **Which models beat the baseline?** Both LinearRegression and Ridge. Their MAE is about 40 % lower than the baseline's (6.9 vs 11.6), and R^2 rises from −0.10 to 0.59. The baseline's R^2 is slightly negative because the test-set mean differs a

little from the training mean.

2. **Is the difference large enough to matter?** Yes compared with the baseline (roughly 5 percentage points better on average). The difference between LinearRegression and Ridge is tiny and does not matter.
3. **Most understandable metric:** MAE, because it is in quiz percentage points. The model is wrong by about 7 points on average.
4. **Unusually large errors?** A few. RMSE (8.4) is larger than MAE (6.9), and the largest errors are about 15–19 points (about 6 % of test rows are off by more than 15 points).

3. Residuals and visualization

In [10]:
```
resid = y_test.values - lr_pred
fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
ax[0].scatter(y_test, lr_pred, s=20); ax[0].plot([15, 100], [15, 100], "k--")
ax[0].set_xlabel("actual next-week score"); ax[0].set_ylabel("predicted"); ax[0].set_title("Actual vs predicted")
ax[1].scatter(lr_pred, resid, s=20); ax[1].axhline(0, color="k", ls="--")
ax[1].set_xlabel("predicted"); ax[1].set_ylabel("residual (actual - predicted)"); ax[1].set_title("Residual plot")
plt.tight_layout(); plt.show()
```



- The points follow the diagonal line, but with a clear spread.
- The residuals look mostly random around 0, and there is no strong curve.
- The model tends to **over-predict very low scores and under-predict very high scores**. The predictions are pulled towards the middle, which is typical for linear regression with noisy data.
- A few points have large residuals (about ±15–19), which could be unusual weeks that the features cannot explain.

4. Validation and split sensitivity

In [11]:
```
from sklearn.model_selection import GroupShuffleSplit
rows = []
for seed in [0, 1, 2, 3, 4]:
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=seed)
    for name, m in [("Dummy", DummyRegressor()), ("Linear", LinearRegression()), ("Ridge", Ridge())]:
        p = Pipeline([("prep", prep), ("model", m)]).fit(Xtr, ytr).predict(Xte)
        rows.append({"split": f"random {seed}", "model": name, "MAE": mean_absolute_error(yte, p),
                     "RMSE": root_mean_squared_error(yte, p), "R2": r2_score(yte, p)})
# split by student: no student in both train and test
gss = GroupShuffleSplit(n_splits=5, test_size=0.25, random_state=0)
for i, (tr, te) in enumerate(gss.split(X, y, groups=df["student_key"])):
    p = Pipeline([("prep", prep), ("model", LinearRegression())]).fit(X.iloc[tr], y.iloc[tr]).predict(X.iloc[te])
    rows.append({"split": f"by student {i}", "model": "Linear", "MAE": mean_absolute_error(y.iloc[te], p),
                 "RMSE": root_mean_squared_error(y.iloc[te], p), "R2": r2_score(y.iloc[te], p)})
val = pd.DataFrame(rows)
val["type"] = val["split"].str.rsplit(" ", n=1).str[0]
val.groupby(["type", "model"])[["MAE", "RMSE", "R2"]].agg(["mean", "min", "max"]).round(2)
```

		MAE			RMSE			R2		
		mean	min	max	mean	min	max	mean	min	max
type	model									
by student	Linear	6.88	6.30	7.49	8.58	7.85	9.34	0.63	0.50	0.71
random	Dummy	11.68	9.63	13.21	14.54	11.83	16.40	-0.02	-0.06	-0.00
	Linear	6.66	6.20	7.20	8.15	7.51	8.53	0.67	0.51	0.73
	Ridge	6.66	6.20	7.22	8.15	7.50	8.54	0.67	0.51	0.73

- Over 5 random splits, the MAE of linear regression varies between 6.2 and 7.2 and R^2 between 0.51 and 0.73, but it **always beats the baseline clearly** (baseline MAE 9.6–13.2). The main conclusion does not change.
- Linear and Ridge stay practically equal in every split.

- **Repeated observations:** each student has 7 rows, so a random row split puts the same student in both training and test. Splitting **by student** (GroupShuffleSplit) gives similar results here (MAE 6.3–7.5), so the optimism is small in this dataset. For a real project, a split by student or by time would be more defensible.

5. Data leakage

In [12]:
```
X_leak = df[num_features + cat_features + ["next_week_quiz_grade_0_5"]]
Xtr, Xte, ytr, yte = train_test_split(X_leak, y, test_size=0.25, random_state=42)
prep_leak = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())]), num_features +
["next_week_quiz_grade_0_5"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_features)])
p = Pipeline([("prep", prep_leak), ("model", LinearRegression())]).fit(Xtr, ytr).predict(Xte)
print("With leaked grade -> MAE:", round(mean_absolute_error(yte, p), 2), " R²:", round(r2_score(yte, p), 3))
```

```
With leaked grade -> MAE: 4.36  R²: 0.834
```

Adding `next_week_quiz_grade_0_5` makes the results suspiciously good (MAE drops from 6.9 to 4.4 and R² rises from 0.59 to 0.83). This grade is calculated from next week's score, the very value we want to predict, so it would not exist at the prediction point. All preprocessing is inside a pipeline and fitted on the training data only.

Interpretation and limitations

- Earlier quiz results are the strongest predictors of next week's score, and the linear model is about 40 % better than always guessing the mean.
- The typical error is still about 7 percentage points, which is enough to misjudge individual students. It is fine for rough trends, not for decisions about one person.
- Limitations: synthetic data, a small dataset (280 rows from 40 students), repeated rows per student, a straight-line model, and coefficients that show association, not cause.

Friday – Practical verification values

In [13]:
```
print("Observations:", len(df), "| Target:", target, "| Features:", len(num_features) + len(cat_features))
print("Train / test:", len(X_train), "/", len(X_test))
results[["Model", "MAE", "RMSE", "R²"]]
```

```
Observations: 280 | Target: next_week_quiz_score_pct | Features: 11
Train / test: 210 / 70
```

	Model	MAE	RMSE	R²
0	DummyRegressor	11.62	13.72	-0.100
1	LinearRegression	6.88	8.35	0.592
2	Ridge	6.89	8.36	0.591

Friday – Concept map

